

- *Java/Node.js* для построения серверной части;
- *PostgreSQL* (с использованием *JSONB* для гибкого хранения метаданных);
- *Docker*, обеспечивающий переносимость и стабильность программной среды;
- *S3*-совместимые объектные хранилища для медиаконтента;
- *RESTful API* для взаимодействия клиента и сервера.

Дипломный проект выполнен самостоятельно, проверен в системе «Антиплагиат». Процент оригинальности соответствует норме, установленной кафедрой. Цитирования обозначены ссылками на публикации, указанные в «Списке использованных источников»

Дипломный проект выполнен самостоятельно, проверен в системе «Антиплагиат». Процент оригинальности соответствует норме, установленной кафедрой. Цитирования обозначены ссылками на публикации, указанные в «Списке использованных источников».

1 ОПИСАНИЕ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Структура системы управления и виды деятельности

БГУИР сегодня – это крупный учебно-научно-инновационный комплекс, в структуру которого входят 8 факультетов: компьютерного проектирования (ФКП), информационных технологий и управления (ФИТУ), радиотехники и электроники (ФРЭ), компьютерных систем и сетей (ФКСиС), информационной безопасности (ФИБ), инженерно-экономический (ИЭФ), военный факультет (ВФ), факультет доуниверситетской подготовки и профессиональной ориентации (ФДППО) [1].

Университет включает в себя 9 факультетов, 32 кафедры, институт информационных технологий. Филиалом университета является Минский радиотехнический колледж. БГУИР осуществляет подготовку студентов по 41 специальности первой ступени высшего образования, по 15 – второй ступени, по 30 – аспирантуры и 15 – докторантуры.

На балансе университета находится 8 учебно-практических корпусов и 5 общежитий. При этом они не формируют кампуса, так как расположены в нескольких местах небольшими группами. Большинство зданий университета построены в 1960-х – 1980-х годах в стиле советского архитектурного модернизма.

Основные виды деятельности БГУИР:

1 Образовательная деятельность – подготовка высококвалифицированных ИТ-специалистов, инженеров по радиоэлектронике, микроэлектронике и информационной безопасности на всех уровнях: бакалавриат, магистратура и аспирантура.

2 Научная и инновационная деятельность – проведение фундаментальных и прикладных исследований в области искусственного интеллекта, кибербезопасности, нанотехнологий и радиотехники, а также внедрение разработок в реальный сектор экономики.

3 Международное сотрудничество – активное взаимодействие с ведущими зарубежными техническими вузами и мировыми ИТ-гигантами, реализация программ «двойных дипломов» и участие в международных образовательных и научных грантах.

4 Взаимодействие с индустрией и бизнес-образование – создание совместных лабораторий с крупнейшими ИТ-компаниями, проведение сертифицированных курсов (*Cisco*, *SAP*, *Microsoft* и др.) и подготовка кадров под конкретные задачи цифровой экономики.

5 Цифровая трансформация и просвещение – проведение профильных конференций, хакатонов и олимпиад по программированию, направленных на популяризацию инженерного образования и развитие цифровых компетенций в обществе.

6 Техническая экспертиза и консалтинг – предоставление услуг по разработке программного обеспечения, проектированию радиоэлектронных систем, а также проведение экспертиз в области защиты информации.

БГУИР обладает развитой многоуровневой структурой и эффективной системой управления, адаптированной под задачи высокотехнологичного вуза. Структура университета объединяет факультеты, общеуниверситетские кафедры, научно-исследовательские части, отраслевые лаборатории, центры трансфера технологий, библиотеку, издательство и мощный центр информационных технологий.

Во главе БГУИР стоит ректор, который осуществляет общее руководство стратегическим развитием и всеми направлениями деятельности вуза. Команда проректоров курирует ключевые блоки: учебную работу (организация учебных планов, аттестация), научную деятельность (инновационные разработки, аспирантура) и воспитательную работу. Деканы факультетов управляют отдельными подразделениями, отвечая за качество подготовки инженеров, подбор преподавательского состава и связь с профильными предприятиями-заказчиками кадров.

Кафедры являются базовыми звеньями, обеспечивающими проведение занятий по техническим и гуманитарным дисциплинам. Они также отвечают за работу совместных учебных лабораторий с IT-компаниями, выполнение научно-исследовательских и опытно-конструкторских работ (НИОКР) и актуализацию учебных программ под требования рынка. В университете активно действует система студенческого самоуправления, представленная студенческим советом, профкомом и молодежными центрами, которые организуют хакатоны, технические конкурсы, культурные фестивали и волонтерские проекты.

Фундаментальным элементом системы управления БГУИР является его цифровая экосистема. Она включает в себя передовую информационную инфраструктуру: систему «Электронный университет», личные кабинеты студентов и преподавателей, электронные ведомости, репозиторий научных трудов и платформы дистанционного обучения. Это позволяет автоматизировать большинство управленческих процессов и обеспечить прозрачность контроля успеваемости.

Одной из главных задач системы управления БГУИР является создание инновационной образовательной среды, стимулирующей научно-техническое

творчество, обеспечение лидерства в сфере цифровых технологий, а также укрепление престижа университета как ведущего центра подготовки кадров для «экономики знаний».

Прохождение преддипломной практики осуществлялось в отделе автоматизированных систем управления и информационных технологий. Основные направления деятельности отдела [2]:

- 1 Совершенствование процесса управления университетом и обучения на основе применения автоматизированных систем управления (АСУ). Использования современных средств телекоммуникации и информационных технологий.

- 2 Внедрение и использование АСУ в управлении образовательным процессом.

- 3 Организация электронного документооборота структурных подразделений университета.

- 4 Разработка и внедрение в образовательный процесс новых образовательных информационных технологий.

- 5 Разработка сопровождающего программного обеспечения для электронных учебно-методических комплексов.

- 6 Разработка и администрирование тестирующих программ для контроля знаний.

1.2 Информационные системы

БГУИР активно использует различные информационные системы в своей деятельности, направленные на автоматизацию управления и учебного процесса.

Одной из ключевых составляющих цифровой экосистемы БГУИР является Интегрированная информационная система (ИИС). Это уникальная разработка университета, объединяющая в себе функции управления учебным процессом, личного кабинета студента и преподавателя, а также инструменты электронного документооборота.

ИИС БГУИР представляет собой единое цифровое пространство, которое автоматизирует взаимодействие всех участников образовательного процесса. Основные возможности системы включают:

- 1 Личный кабинет студента. Здесь доступно актуальное электронное расписание, зачетная книжка с оценками за все семестры, а также информация о назначенных стипендиях и пропусках занятий.

планировать гастроли, опираясь на жесткие цифры из панели управления сервисом.

Теперь проанализируем аналогичные системы: *Spotify* и Яндекс Музыка.

Spotify в современной ИТ-индустрии считается золотым стандартом распределенных систем, который превратил прослушивание музыки из линейного процесса в высокотехнологичный сервис на основе данных. В отличие от традиционных плееров, архитектура этого сервиса построена на принципах максимальной автономности и масштабируемости, что позволяет ему обслуживать сотни миллионов пользователей без видимых задержек.

Техническое устройство *Spotify* базируется на идее разделения монолитного приложения на сотни независимых микросервисов. Каждый такой сервис отвечает за строго определенную операцию: от обработки поискового запроса до генерации обложки плейлиста. Это позволяет системе быть невероятно отказоустойчивой – если у сервиса ломается модуль социальных связей, пользователь все равно сможет слушать музыку. В дипломе такую структуру можно описать как модульную архитектуру, где каждый компонент взаимодействует с другими через стандартизированные программные интерфейсы (*API*), обеспечивая гибкость всей системы.

Одной из главных инженерных побед *Spotify* стала разработка проприетарных протоколов передачи данных. Чтобы музыка начинала играть мгновенно после нажатия кнопки, сервис использует агрессивное кэширование и предсказание поведения. Приложение анализирует, какой трек в плейлисте идет следующим, и начинает подгружать его начало в фоновом режиме еще до того, как текущая песня закончится. На системном уровне это реализуется через сложную иерархию серверов: популярные треки хранятся «ближе» к пользователю в оперативной памяти граничных серверов, тогда как редкие записи лежат в глубоких облачных хранилищах.

Сердце *Spotify* – это его рекомендательная модель, которая фактически является самой сложнейшей системой обработки сигналов и анализа текстов. Сервис не просто группирует музыку по жанрам, он создает многомерное векторное пространство, где каждый трек имеет сотни характеристик: от акустичности и танцевальности до «энергии» звука. Система постоянно сопоставляет профили прослушивания разных людей, находя неочевидные закономерности. Если тысячи пользователей с похожими вкусами внезапно начали слушать нового исполнителя, алгоритм автоматически поднимет его приоритет в твоих персональных подборках. Для АИС это означает наличие мощного аналитического модуля, работающего с нейронными сетями в реальном времени.

Spotify генерирует колоссальный объем технических событий в секунду. Каждое нажатие на паузу, пропуск трека или изменение громкости отправляется в централизованную систему сбора данных. Эти логи используются не только для статистики, но и для обучения моделей машинного обучения. Внутренняя инфраструктура сервиса спроектирована так, чтобы переваривать эти терабайты данных, превращая сырые логи в осмысленные отчеты для лейблов и персональные итоги года для слушателей. В контексте дипломной работы это можно представить как систему событийного мониторинга, которая связывает активность пользователя с бизнес-логикой сервиса.

Уникальной чертой системы является функция *Connect*, которая превращает все устройства пользователя в единую управляемую сеть. Это реализуется через поддержку постоянного веб-сокета соединения между клиентом и сервером. Когда ты переключаешь трек на компьютере с помощью телефона, сигнал проходит через облачный координатор, который мгновенно обновляет состояние плеера на всех активных сессиях. Такая бесшовная интеграция требует сложной логики синхронизации состояний в базе данных, чтобы избежать конфликтов при одновременном доступе с разных устройств.

Интерфейс *Spotify* – это эталон акцентного минимализма и карточной архитектуры. В 2026 году он окончательно закрепил за собой статус «умного черного зеркала», где дизайн не отвлекает от контента, а подстраивается под него.

Дизайн интерфейса *Spotify* представляет собой современную многопанельную структуру, построенную на принципах глубокой темной темы, где визуальный контент преобладает над текстовым, как представлено на рисунке 1.1. Основное пространство разделено на функциональные зоны, которые обеспечивают бесшовную навигацию без необходимости перезагрузки страниц. Левая узкая панель служит быстрым доступом к личной медиатеке, используя компактные квадратные пиктограммы обложек, что позволяет экономить рабочее место. Центральная область отдана под динамическую сетку рекомендаций и недавних прослушиваний, где крупные плитки с мягким скруглением углов и контрастным текстом позволяют пользователю мгновенно ориентироваться в контенте.

Особое внимание уделено правой информационной панели, которая акцентирует внимание на текущем треке через полноразмерное изображение обложки альбома, создавая глубокое визуальное погружение. Нижняя часть интерфейса выделена под фиксированную панель управления воспроизведением, которая визуально отделена от основного контента. Здесь используются интуитивно понятные иконки управления, шкала прогресса и

дополнительные инструменты, такие как управление очередью и громкостью. Цветовая палитра базируется на сочетании радикально черного и темно-серого фонов, на которых ярко выделяются акцентные элементы – например, зеленый цвет активных кнопок или индикаторов воспроизведения. Такой подход минимизирует нагрузку на зрение при длительном использовании и выводит на первый план художественное оформление музыкальных релизов, превращая интерфейс в эстетичное дополнение к аудиопотоку.

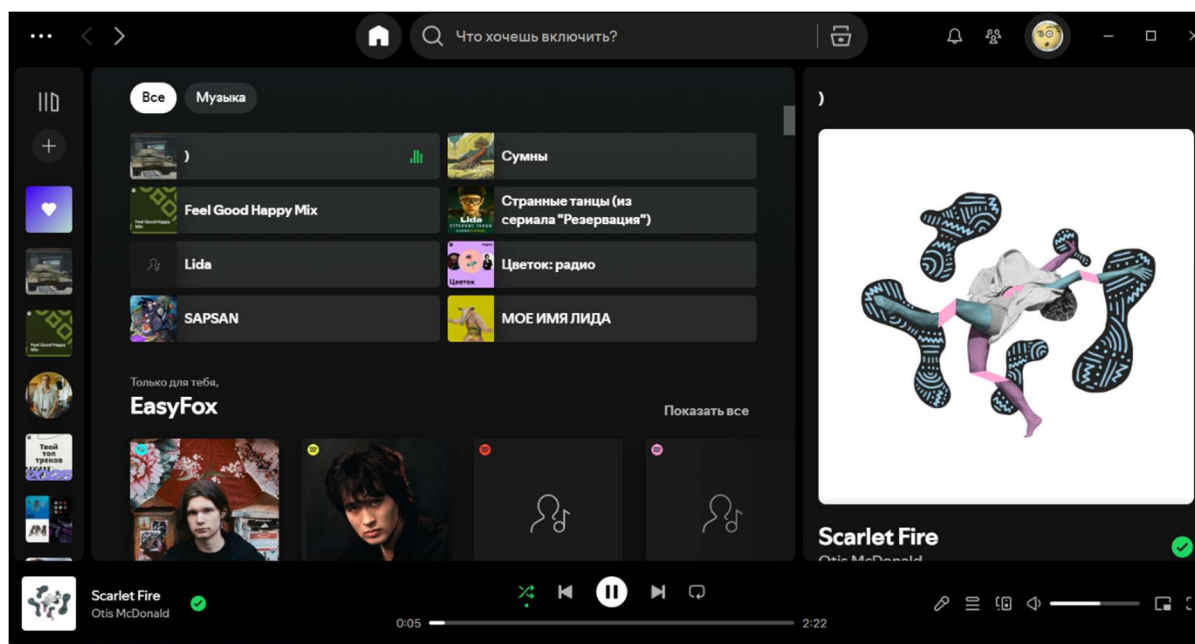


Рисунок 1.1 – Главная страница *Spotify*

Интерфейс мобильного приложения *Spotify*, представленный на рисунке 1.2, демонстрирует современную концепцию вертикально-ориентированного адаптивного дизайна, где ключевым приоритетом является удобство управления одной рукой. Визуальная иерархия выстроена таким образом, что наиболее актуальный контент располагается в верхней трети экрана, обеспечивая мгновенный визуальный контакт с последними действиями пользователя. Цветовое решение базируется на глубоком темном фоне, что позволяет эффективно использовать контраст для выделения интерактивных элементов, таких как ярко-зеленая кнопка фильтрации контента, служащая главным цветовым акцентом верхней панели.

Центральная часть экрана организована в виде сетки из восьми компактных карточек с закругленными углами, которые объединяют графический контент и текстовые метаданные в единый навигационный блок. Такой подход минимизирует когнитивную нагрузку, позволяя пользователю считывать информацию через узнаваемые обложки артистов и плейлистов.

Ниже располагается секция персональных рекомендаций, использующая более крупные квадратные плитки с градиентной заливкой, что визуальнo отделяет алгоритмически созданный контент от истории прослушиваний и акцентирует внимание на функциях социального взаимодействия, таких как создание общих плейлистов.

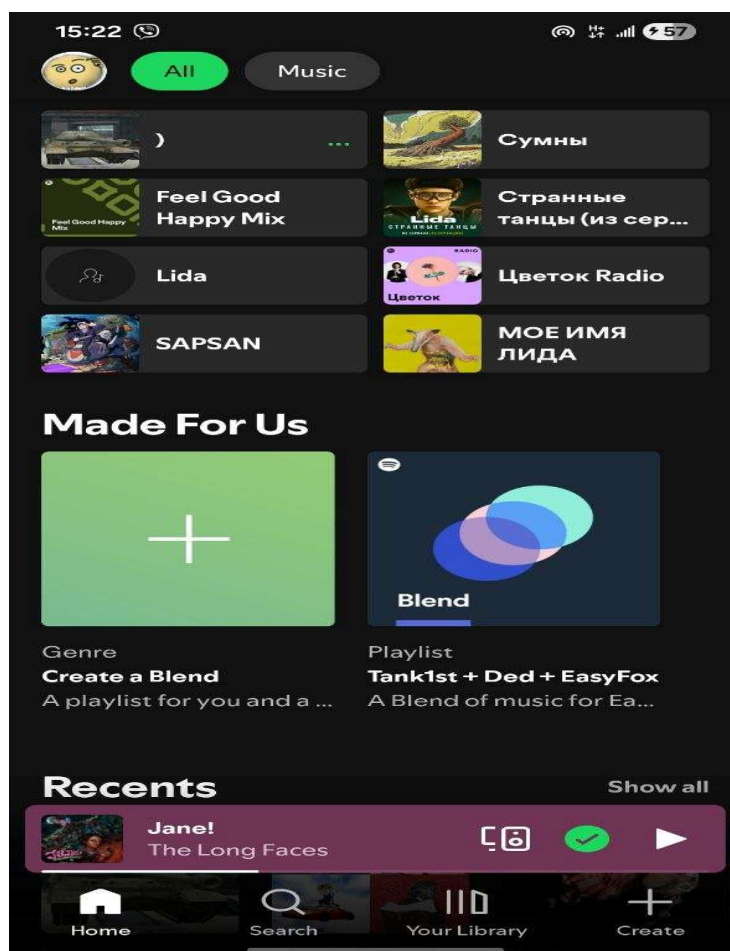


Рисунок 1.2 – Главная страница мобильного *Spotify*

Особое значение в архитектуре интерфейса имеет нижняя навигационная панель и плавающий мини-плеер. Плеер визуальнo выделен мягким вишневым оттенком, извлеченным из палитры обложки текущего трека, что создает эффект динамического интерфейса, подстраивающегося под контекст прослушивания. Панель управления в самом низу экрана обеспечивает быстрый доступ к ключевым разделам системы – поиску, библиотеке и созданию контента – через минималистичные контурные иконки. Общая композиция элементов стремится к максимальной информативности при сохранении «воздуха» в интерфейсе, что характерно для современных

АИС мультимедийной направленности, где дизайн служит лишь проводником к основному контенту.

Яндекс Музыка – это пример того, как классический музыкальный сервис трансформировался в продукт, полностью управляемый искусственным интеллектом. Если *Spotify* делает ставку на социальные связи и кураторские плейлисты, то Яндекс пошел по пути максимальной автоматизации и «бесшовного» потребления контента внутри своей экосистемы.

Ключевое отличие Яндекса заключается в смещении фокуса с библиотеки (где пользователь ищет альбом) на «поток» (где система сама решает, что играть). Центральным элементом АИС здесь является система «Моя волна». С точки зрения системного анализа, это бесконечный генератор очереди воспроизведения, который пересчитывает приоритеты выдачи треков после каждого действия пользователя: лайка, пропуска или даже изменения громкости. Это делает систему не статичным хранилищем, а динамическим процессом.

Технологический стек Яндекса опирается на две мощные группы алгоритмов, работающих в связке:

- 1 Коллаборативная фильтрация. Система анализирует миллиарды взаимодействий миллионов пользователей. Она ищет «цифровых близнецов» – людей с идентичными паттернами поведения – и предлагает треки, которые зашли им, но еще не прослушаны вами.

- 2 Глубокий анализ аудиоконтента (*Deep Audio Analysis*): Нейросети Яндекса «слушают» спектрограмму каждого трека. Они выделяют тембр голоса, бит, ритм и даже эмоциональный окрас (настроение). Это позволяет системе подмешивать в поток абсолютно новых, неизвестных артистов только потому, что их звучание математически схоже с тем, что любит пользователь.

Яндекс Музыка уникальна своей глубокой интеграцией с другими сервисами (Алиса, Навигатор, Яндекс Книги). АИС получает данные о контексте пользователя: если музыка запущена через умную колонку вечером, система отдаст приоритет спокойным трекам; если в машине через Навигатор – динамичным. Технически это реализуется через единый идентификатор (*Yandex ID*) и общую шину данных, которая передает контекстные теги в рекомендательный движок.

Интересной инженерной особенностью является динамическая визуализация (цветовая индикация «Моей волны»). Это не просто анимация, а результат работы алгоритма, который на лету анализирует частотные характеристики проигрываемого аудиопотока и сопоставляет их с метаданными настроения трека.

Внутри системы развернут мощный аналитический блок для правообладателей. Яндекс предоставляет артистам доступ к данным в реальном времени: они видят, как их треки попадают в рекомендации, каков процент дослушиваний и как география слушателей влияет на популярность. С точки зрения АИС – это сложная система ролевого доступа к базам данных *Big Data*, где сырые логи прослушиваний агрегируются в понятные бизнес-метрики.

Интерфейс Яндекс Музыки на представленном рисунке 1.3 демонстрирует современный подход к проектированию стриминговых платформ, где ключевой акцент сделан на функциональном зонировании и алгоритмической подаче контента. Дизайн выполнен в темной цветовой гамме, которая служит нейтральным фоном для ярких интерактивных элементов и обложек, не отвлекая пользователя от основной задачи – выбора музыки.

Левая вертикальная панель представляет собой классическое навигационное меню с интуитивно понятными пиктограммами и текстовыми подписями, что обеспечивает быстрый переход между глобальными разделами сервиса. В нижней части этой панели расположен блок авторизации пользователя и кнопка загрузки десктопного приложения, что подчеркивает экосистемный характер продукта.

Центральное рабочее пространство организовано по принципу горизонтальных лент (каруселей). В верхней части расположены приоритетные блоки «Избранное» и «История», оформленные в виде крупных карточек с мягким градиентом, что выделяет их на общем фоне. Ниже представлен уникальный блок «Моя волна», реализованный через систему тегов-фильтров (по жанру, настроению, активности). Это позволяет пользователю кастомизировать поток воспроизведения в один клик. Визуальное решение этих кнопок выполнено с использованием абстрактных паттернов, что подчеркивает «интеллектуальность» и динамичность алгоритмов.

Нижнюю часть интерфейса занимает фиксированный плеер, который является центром управления звуком. Он визуально отделен от контентной области тонкой разделительной линией и содержит все необходимые инструменты: индикатор прогресса трека, кнопки управления очередью, лайк и настройки громкости. Яркая желтая кнопка воспроизведения в центре плеера служит главным визуальным якорем всего интерфейса. Дизайн в целом выглядит цельным и сбалансированным, сочетая в себе строгость системных компонентов с яркостью и эмоциональностью музыкального контента.

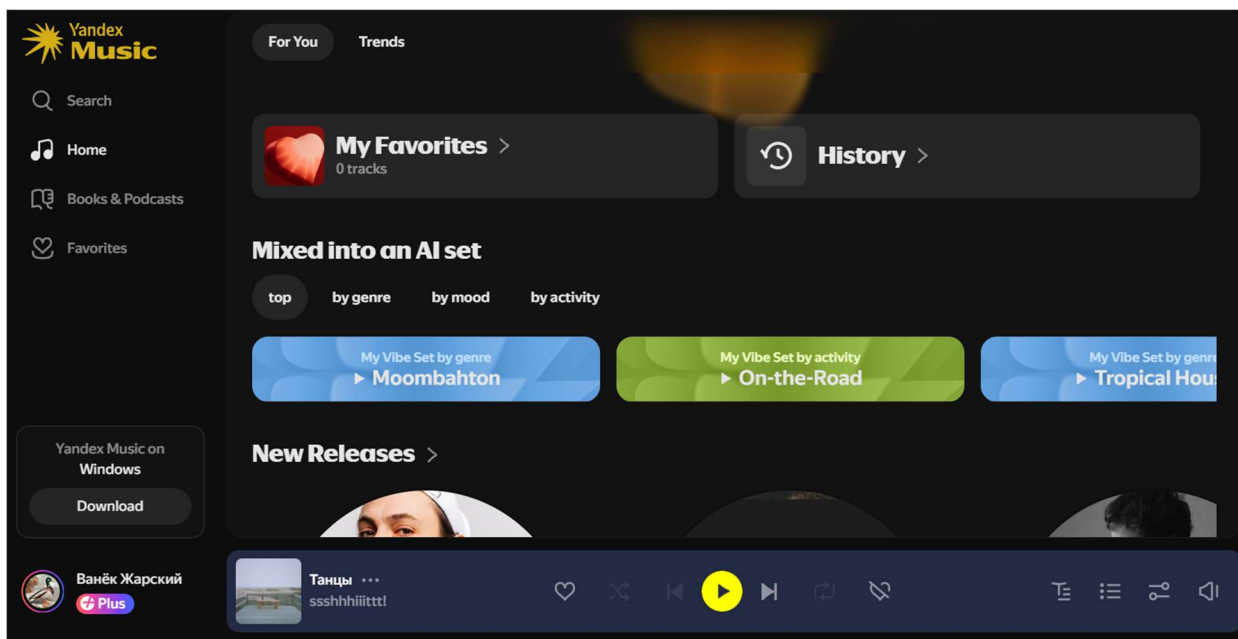


Рисунок 1.3 – Главная страница Яндекс Музыка

YouTube Music – это уникальный гибрид классического стриминга и крупнейшего в мире видеохостинга. Если *Spotify* – это «умный алгоритм», а Яндекс – «бесконечный поток», то *YouTube Music* – это «всеобъемлющий контекст».

Главное техническое преимущество *YouTube Music* – доступ к поисковой истории всей экосистемы *Google*. АИС сервиса знает не только то, что ты слушал, но и что ты искал в *Google* или смотрел на основном *YouTube*. Это позволяет системе строить «сверхпрофиль» пользователя.

В отличие от конкурентов, база данных *YouTube Music* не ограничена только официальными релизами от лейблов.

Система индексирует миллиарды видеороликов. Благодаря алгоритмам компьютерного зрения и анализа звуковых отпечатков (*Content ID*), сервис вычленяет аудиодорожки из видео (каверы, лайвы с концертов, ремиксы, фанатские нарезки) и бесшовно встраивает их в музыкальную библиотеку.

Это пример системы с неструктурированными источниками данных, где механизм фильтрации и классификации (отсеивание плохого звука от студийного) является ключевым бизнес-процессом.

YouTube Music делает ставку на «здесь и сейчас». Используя данные о геолокации и активности пользователя, система меняет интерфейс в реальном времени. Если АИС видит, что пользователь находится в спортзале (по *GPS* и истории посещений), она выводит на главный экран энергичные плейлисты. Если пользователь в аэропорту – подборки для путешествий. Это реализуется

через динамическую генерацию интерфейса на основе контекстных метаданных, что является отличной темой для отдельной главы диплома.

Одной из сложнейших инженерных задач, решенных в *YouTube Music*, является мгновенное переключение между аудиодорожкой и видеоклипом без прерывания звука.

Система синхронизирует два разных потока данных (аудиофайл высокого качества и видеофайл) по таймкодам с точностью до миллисекунды.

Это можно описать как алгоритм синхронизации мультимедийных потоков из гетерогенных источников, что подчеркивает технологическую сложность архитектуры.

Поисковый движок *YouTube Music* – один из самых мощных в индустрии, так как он наследует технологии *Google* Поиска. Пользователь может вбить «песня, где парень поет в пустыне про любовь» или часть невнятного текста, и система найдет трек. Это пример внедрения *NLP* (обработки естественного языка) и семантического поиска в музыкальную АИС, что позволяет находить контент даже при отсутствии точных метаданных.

YouTube Music – это пример того, как музыкальный сервис может использовать внешние данные (видео, поиск, локация) для создания максимально точного пользовательского опыта.

Интерфейс *YouTube* «Музыка», представленный на рисунке 1.4, отражает философию масштабируемости и интеграции, где музыкальный сервис является частью глобального видеохостинга. В отличие от *Spotify* или Яндекс Музыки, здесь дизайн строится на строгой блочной системе, которая должна одинаково эффективно отображать как профессиональные студийные альбомы, так и пользовательский видеоконтент. Визуальная доминанта смещена в сторону максимально крупных превью-карточек, которые занимают почти всё полезное пространство центральной части экрана, создавая эффект витрины.

Левая навигационная панель выполнена в классическом стиле сервисов *Google*: она максимально лаконична и использует тонкие контурные иконки на темном фоне. Это позволяет пользователю быстро переключаться между основным *YouTube* и его музыкальным подsegmentом, сохраняя единообразие пользовательского опыта. В верхней части интерфейса находится массивная строка поиска, которая визуально объединена с кнопками голосового ввода и создания контента, что подчеркивает активную роль пользователя в системе – он не только потребитель, но и потенциальный автор.

Особое внимание в дизайне уделено карточкам плейлистов в ленте рекомендаций. Каждая плитка имеет вертикальную ориентацию и включает в себя яркое изображение с наложенным поверх него контрастным

типографическим оформлением названия подборки. В нижней части каждой карточки расположен полупрозрачный индикатор количества треков, что добавляет интерфейсу информативности без перегрузки деталями. Использование радикально черного фона в сочетании с яркими, насыщенными цветами обложек создает динамичную и современную эстетику, которая ориентирована на визуальное восприятие и быстрый скроллинг в поисках нужного настроения [3].

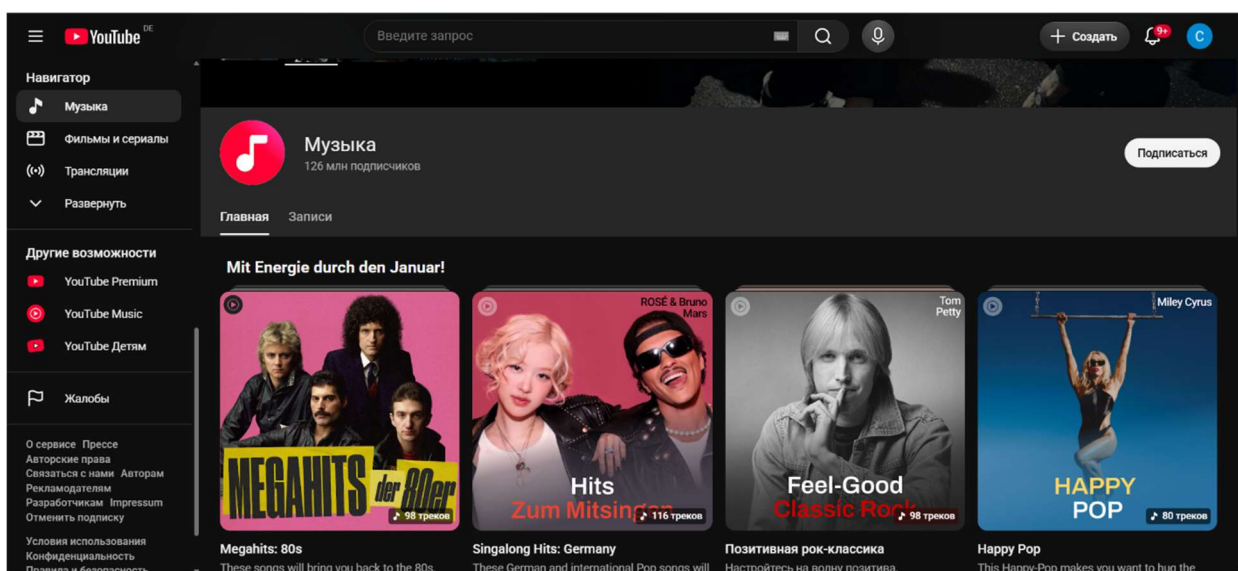


Рисунок 1.4 – Главная страница *YouTube Music*

1.4 Постановка задачи дипломного проекта

В настоящее время многие независимые музыкальные лейблы и современные медиа-платформы не имеют собственных автоматизированных информационных систем, которые бы в полной мере отвечали потребностям анализа предпочтений слушателей и эффективного управления медиатекой. Для решения задач по хранению музыкального контента, формированию плейлистов и взаимодействию с аудиторией в данный момент используются разрозненные облачные хранилища и сторонние социальные сети, не обеспечивающие единой структуры данных [4].

При анализе существующих систем-аналогов (таких как *Spotify* или Яндекс.Музыка) определено, что их готовые решения не могут быть интегрированы в частные инфраструктуры из-за закрытости алгоритмов и отсутствия возможности кастомизации под узкие жанровые или корпоративные требования, а качество рекомендательных моделей напрямую влияет на удержание аудитории в рамках конкретного сервиса.

Таким образом, определена задача дипломного проекта: необходимо создать автоматизированную информационную систему музыкального сервиса с модулем персональных рекомендаций для управления аудио-каталогом и обеспечения доступа к контенту как для администраторов (кураторов лейбла), так и для конечных слушателей.

Система должна иметь адаптивный веб-интерфейс, созданный с учетом требований к юзабилити современных мультимедийных приложений и специфики потребления аудио-контента.

Кроме того, определены исходные данные к проекту. Система должна корректно работать в таких браузерах как *Google Chrome* (версия 90 и выше), *Mozilla Firefox* (версия 85 и выше), *Safari* (версия 14 и выше), *Microsoft Edge* (версия 90 и выше). Используемые технологии: язык программирования – *Java* для серверной части, *JavaScript (React.js)*, *HTML5*, *CSS3* для клиентской части; система управления базами данных – *PostgreSQL*; хранение медиа-файлов – объектное хранилище (*S3*-совместимое).

2 ПРОЕКТИРОВАНИЕ СТРУКТУРЫ ИНФОРМАЦИОННОЙ СИСТЕМЫ

2.1 Организационно-экономическая сущность задачи

Назначением разрабатываемой АИС является автоматизация процессов управления музыкальным контентом, хранения медиа-активов и персонализированной доставки контента конечным пользователям (слушателям). Система призвана решить проблему отсутствия единой инфраструктуры управления для независимых медиа-платформ, минимизировать трудозатраты на ручную обработку метаданных и обеспечить масштабируемость при увеличении объема медиатеки. Внедрение системы позволяет перейти от разрозненных методов хранения (облачные диски, неструктурированные базы) к централизованному управлению данными, что существенно повышает эффективность деятельности лейбла или стриминговой площадки.

Система реализует следующие основные функции:

- 1 Централизованное управление контентом. Загрузка, редактирование метаданных и классификация аудиофайлов по жанрам, артистам и альбомам.
- 2 Интеллектуальная рекомендательная подсистема. Автоматическое формирование очереди воспроизведения на основе анализа предпочтений пользователя и акустических характеристик треков.
- 3 Пользовательский стриминг. Обеспечение бесперебойной передачи аудиоданных с использованием адаптивного битрейта для минимизации задержек.
- 4 Аналитическая отчетность. Учет статистики прослушиваний для расчета роялти правообладателям и оценки популярности релизов.

Режимы работы ИС включают:

- 1 Оперативный (*Online*) режим. Обработка запросов пользователей в режиме реального времени (воспроизведение, поиск, навигация по каталогу).
- 2 Пакетный (*Batch*) режим. Выполнение регламентных заданий по обновлению рекомендательных моделей, агрегации статистики за отчетные периоды и созданию бэкапов.

В качестве входной информации выступают:

- 1 Данные от администраторов/артистов. Аудиофайлы в формате *lossless/compressed*, метаданные релизов, обложки (графические файлы).
- 2 Данные от слушателей. Параметры профиля, история действий (лайки, пропуски треков, время прослушивания), поисковые запросы.

музыкального контента и обеспечение высокой степени персонализации для каждого слушателя. Процесс функционирования данного модуля начинается с этапа сбора и профилирования данных, в ходе которого осуществляется детальный анализ истории прослушиваний, а также фиксируются предпочтения пользователя, выраженные через взаимодействия с системой, такие как лайки или пропуски треков. Параллельно с обработкой поведенческих факторов система выполняет анализ аудио-контента, включающий в себя систематизацию метаданных и извлечение ключевых акустических признаков, что позволяет глубже понимать природу предлагаемых произведений. Полученная информация становится базой для генерации кандидатов, где на данном этапе применяются алгоритмы коллаборативной и контентной фильтрации, позволяющие сформировать широкий предварительный список потенциально интересных композиций. Завершающим процессом в работе модуля является финальное ранжирование, в рамках которого происходит присвоение весов релевантности и оценка качества рекомендаций, чтобы сформировать итоговую выдачу, максимально соответствующую актуальным интересам пользователя, что в совокупности создает адаптивный механизм управления медиапоток.

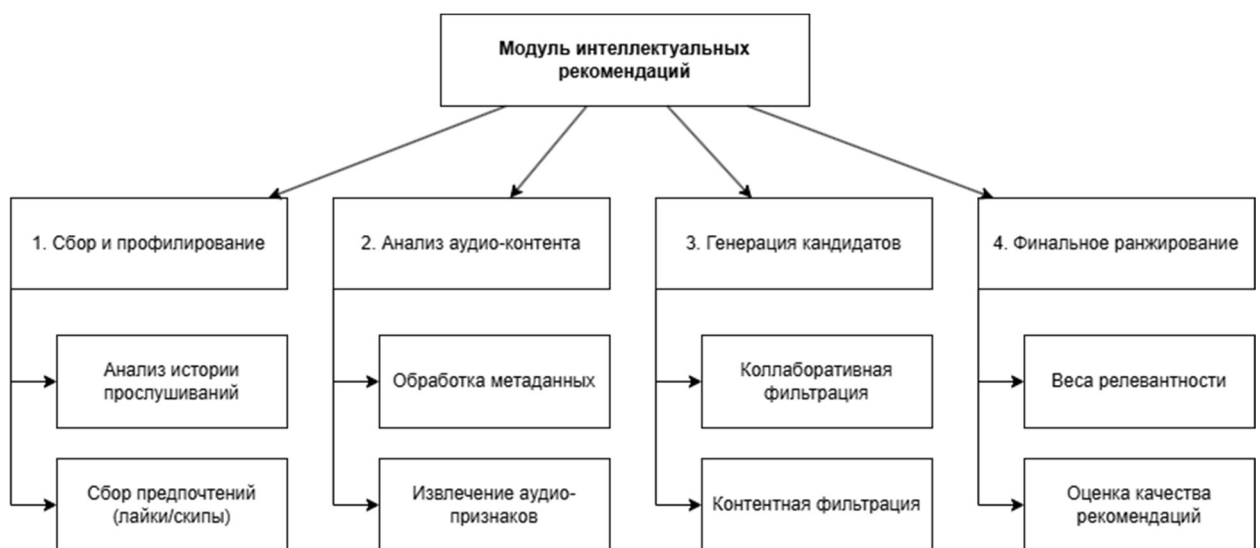


Рисунок 2.1 – Структура модуля интеллектуальных рекомендаций

2.3 Математическое и алгоритмическое обеспечение

В рамках разрабатываемой ИС используются методы обработки данных, направленные на повышение релевантности поиска и персонализацию контента.

Этот алгоритм гарантирует, что новые треки имеют шанс попасть в чарты наравне со старыми хитами.

Для иллюстрации работы рекомендательного движка приведем логику выбора контента, которая представлена на блок-схеме на рисунке 2.2.

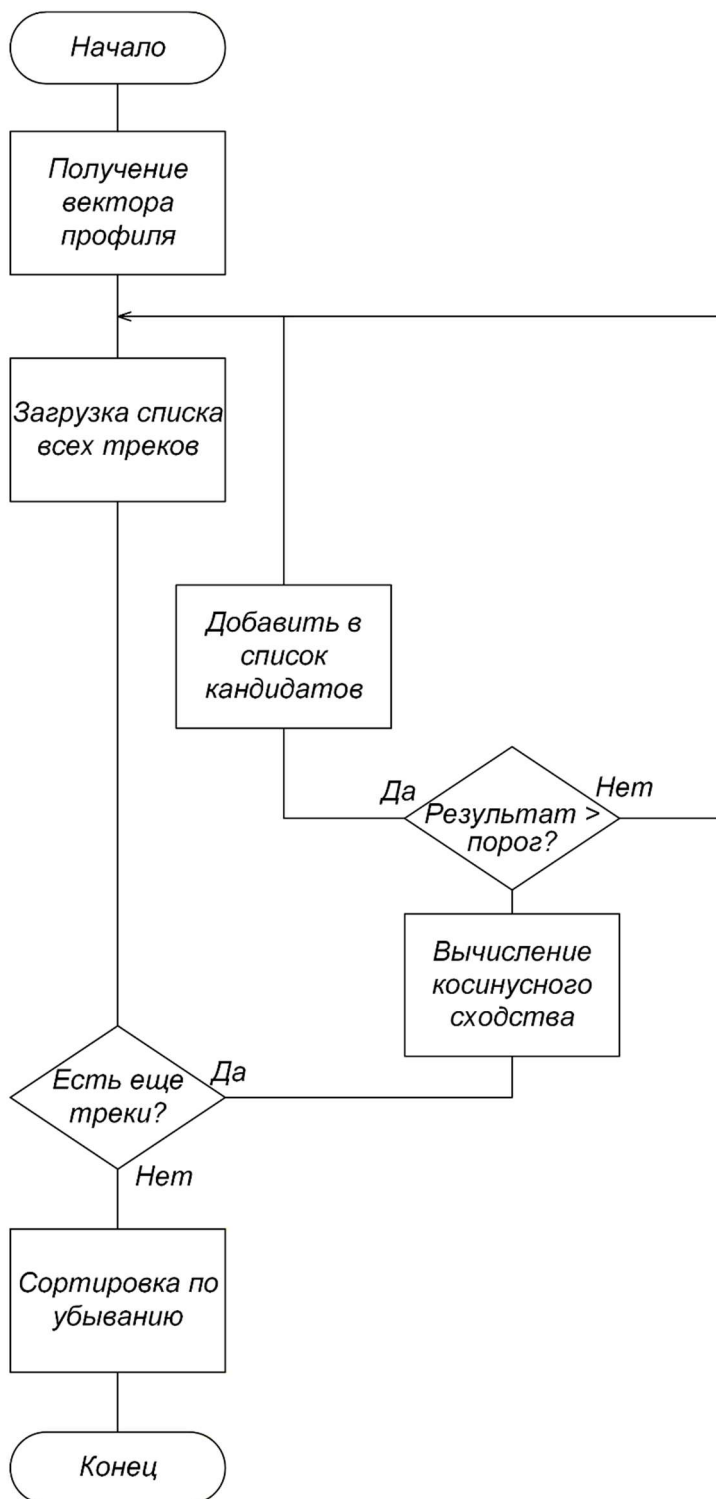


Рисунок 2.2 – Блок-схема алгоритма работы рекомендаций

Таблица 2.4 – Структура таблицы «*tracks*» (Основной контент)

Поле	Тип данных	Описание	Источник данных	Периодичность обновления
<i>track_id</i>	<i>INTEGER</i>	Уникальный идентификатор трека	Автогенерация БД	При создании записи
<i>album_id</i>	<i>INTEGER</i>	Идентификатор альбома	Панель управления артиста	При создании записи
<i>title</i>	<i>VARCHAR(255)</i>	Название композиции	Ввод артистом	При загрузке
<i>duration</i>	<i>INTEGER</i>	Длительность трека (сек.)	Метаданные файла	При загрузке
<i>file_path</i>	<i>VARCHAR(500)</i>	Путь к файлу в S3	Сервер обработки контента	При загрузке
<i>bpm</i>	<i>INTEGER</i>	Темп композиции	Анализатор (алгоритм)	При загрузке

Таблица 2.5 – Структура таблицы «*user_interactions*» (Аналитическая логика)

Поле	Тип данных	Описание	Источник данных	Периодичность обновления
<i>interaction_id</i>	<i>INTEGER</i>	Идентификатор записи взаимодействия	Автогенерация БД	В реальном времени
<i>user_id</i>	<i>INTEGER</i>	Идентификатор пользователя	Сессия авторизации	При действии
<i>track_id</i>	<i>INTEGER</i>	Идентификатор трека	Интерфейс плеера	При действии
<i>interaction_type</i>	<i>VARCHAR(20)</i>	Тип действия (<i>like, listen, skip</i>)	Клиентское приложение	В момент события

Продолжение таблицы 2.5

Поле	Тип данных	Описание	Источник данных	Периодичность обновления
<i>weight</i>	<i>INTEGER</i>	Весовой коэффициент действия	Бизнес-логика системы	При событии
<i>created_at</i>	<i>TIMESTAMP</i>	Время события	Системные часы	В момент события

Таблица 2.6 – Описание связей между таблицами базы данных

Название таблицы 1	Название таблицы 2	Описание
<i>artists</i>	<i>albums</i>	Один исполнитель может выпустить несколько альбомов
<i>albums</i>	<i>tracks</i>	В одном альбоме может содержаться несколько треков
<i>users</i>	<i>playlists</i>	Один пользователь может создать несколько плейлистов
<i>playlists</i>	<i>playlist_tracks</i>	В одном плейлисте может быть несколько записей с треками
<i>tracks</i>	<i>playlist_tracks</i>	Один трек может входить в несколько плейлистов (связь многие-ко-многим)
<i>users</i>	<i>user_interactions</i>	Один пользователь может совершить несколько действий (прослушивание, лайк)
<i>tracks</i>	<i>user_interactions</i>	Один трек может быть связан с множеством действий пользователей
<i>tracks</i>	<i>track_features</i>	Каждому треку соответствует одна запись с аналитическими характеристиками
<i>users</i>	<i>user_profiles</i>	Каждому пользователю соответствует один профиль предпочтений

Таблица 2.7 – Выходные данные, предназначенные для пользователей

Тип данных	Структура (формат)	Получатель	Периодичность	Описание
Результаты поиска	Массив <i>JSON</i> -объектов	Пользователь	При каждом запросе	Список треков/альбомов, соответствующих критериям

Продолжение таблицы 2.7

Тип данных	Структура (формат)	Получатель	Периодичность	Описание
Персональные рекомендации	Массив объектов (<i>ID</i> , метаданные)	Пользователь	При обновлении ленты	Список треков, сформированный алгоритмом
Информация о профиле	Объект JSON	Пользователь	При загрузке страницы	Статистика прослушиваний, история
Статус плейлиста	Булево значение, список треков	Пользователь	В режиме реального времени	Подтверждение сохранения/изменения

Таблица 2.8 – Выходные данные для администратора (отчеты)

Наименование	Получатель	Периодичность	Описание
Отчет об активности	Администратор / Лейбл	Ежемесячно	Статистика по количеству прослушиваний конкретных треков
Журнал системных событий	Администратор	При возникновении критических ошибок	Лог-файл (текстовый формат) для отладки
Статистика пользователей	Менеджер проекта	Еженедельно	Данные о регистрации новых пользователей и их предпочтениях

2.5 Техническое и системное программное обеспечение

Для эффективного функционирования проектируемой информационной системы необходимо наличие комплекса технических и программных средств, обеспечивающих стабильную обработку медиаконтента, выполнение алгоритмов рекомендаций и взаимодействие с пользователями.

Техническая база разделена на серверную часть, обеспечивающую работу ядра системы, и клиентскую часть, используемую для доступа к

функционалу. Требования к техническому обеспечению представлены в таблице 2.9.

Таблица 2.9 – Требования к техническому обеспечению

Компонент системы	Характеристика	Требования
Сервер приложений	Процессор	не менее 4-х ядер, частота 2.4 ГГц и выше
	Оперативная память	не менее 16 Гб (для кэширования БД и обработки запросов)
	Дисковая подсистема	SSD (NVMe) объемом от 500 Гб для БД и логов
Сетевое оборудование	Пропускная способность	Канал связи не менее 1 Гбит/с
Клиентское устройство	Процессор	2-х ядерный, частота 1.8 ГГц и выше
	Оперативная память	не менее 4 Гб
	Монитор	Разрешение не менее 1280x720 пикселей

Программный комплекс включает в себя системное и прикладное программное обеспечение из таблицы 2.10. Выбор технологий обусловлен необходимостью обеспечения высокой надежности, масштабируемости и кроссплатформенности.

Таблица 2.10 – Требования к программному обеспечению

Категория	Наименование	Рекомендуемая версия	Обоснование выбора
1	2	3	4
ОС (сервер)	<i>Ubuntu Server</i> 22.04 LTS	64-bit	Стабильность, поддержка <i>Docker</i>
СУБД	<i>PostgreSQL</i>	16.x	Поддержка сложных реляционных запросов и <i>JSONB</i>

Продолжение таблицы 2.10

1	2	3	4
Среда выполнения	<i>JAVA / Node.js</i>	3.12+ / 20.x	Высокая скорость обработки асинхронных запросов
Контейнеризация	<i>Docker</i>	26.x	Обеспечение переносимости окружения
Хранилище данных	<i>MinIO</i> (S3-совместимое)	Последняя стабильная	Эффективное хранение медиа-объектов
Браузер (клиент)	<i>Chrome/Firefox/Edge</i>	Актуальные версии	Поддержка <i>HTML5</i> и <i>Web Audio API</i>

2.6 Эргономическое обеспечение

При проектировании интерфейса музыкальной информационной системы первоочередное внимание уделялось снижению зрительного утомления и повышению скорости считывания информации[7]. В качестве основного стилистического направления выбрана темная цветовая схема, где фоновые пространства рендерятся в глубоких графитовых и антрацитовых тонах. Это техническое решение минимизирует общую яркость экрана и предотвращает усталость глаз при длительной работе в условиях слабой освещенности, что характерно для сценариев студийной работы или вечернего прослушивания аудиоконтента. Для контрастного выделения текстовых блоков и метаданных используется мягкий белый цвет с контролируемой прозрачностью, варьирующейся в зависимости от иерархической важности элемента. Акцентные интерактивные элементы управления, такие как кнопка запуска воспроизведения и целевая кнопка подтверждения загрузки файлов в модуле Студии, окрашены в насыщенный зеленый цвет, что интуитивно фокусирует внимание пользователя на главных функциях текущего экрана[8].

Шрифтовое оформление интерфейса стандартизировано во всех экранных формах для поддержания стилистического единства. В системе применяется современный гротескный шрифт из семейства без засечек (*Sans-Serif*), характеризующийся высокой читаемостью при малых кеглях. Иерархия текстовых сообщений выстроена за счет четкого разграничения размеров и насыщенности начертания. Названия альбомов, крупных разделов и имена

2 Пользовательский доступ. Осуществляется через авторизованную сессию (*JWT*-токены). Дает право на доступ к личному кабинету, сохранение плейлистов и полнофункциональный стриминг.

3 Административный доступ. Осуществляется через защищенную административную панель с обязательным использованием двухфакторной аутентификации (желательно) или защищенного канала доступа (*VPN/IP-whitelist*).

Таблица 2.11 – Классификация пользователей и права доступа

Роль пользователя	Описание	Основные права
Пользователь (Слушатель)	Зарегистрированный клиент платформы	Просмотр контента, поиск, создание плейлистов, прослушивание треков
Артист (Контент-мейкер)	Автор музыкальных произведений	Загрузка треков, управление метаданными альбомов, просмотр статистики
Администратор	Оператор системы	Управление пользователями, модерация контента, настройка системных параметров, просмотр логов

Таблица 2.12 – Комплекс мер по защите информации

Направление защиты	Мера безопасности	Реализация
1	2	3
Конфиденциальность	Шифрование передаваемых данных	Использование протокола <i>HTTPS (TLS 1.3)</i> для всех запросов
Целостность	Защита от <i>SQL</i> -инъекций	Использование <i>ORM</i> и параметризованных запросов в коде
Аутентификация	Безопасное хранение паролей	Хеширование паролей с использованием алгоритма <i>bcrypt</i> (солирование)

Продолжение таблицы 2.12

1	2	3
Доступ к контенту	Ограничение доступа к файлам	Генерация пре-сайдн (временных) ссылок на S3-объекты
Сохранность	Резервное копирование	Ежедневное создание снимков базы данных и версионирование контента
Мониторинг	Аудит действий	Логирование всех критических операций (вход, изменение прав, удаление контента)

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 Выбор программных средств реализации ИС

Для обеспечения высокой производительности, масштабируемости и модульности разрабатываемой информационной системы (музыкальной стриминговой платформы) выбрана современная многокомпонентная архитектура. Ниже приведено описание и обоснование выбора программных средств для каждого уровня системы.

В качестве основного технологического стека для разработки серверной логики выбран *Java* и фреймворк *Spring Boot*.

Язык программирования *Java* (версия 17/21). Обеспечивает строгую типизацию, платформонезависимость и высокую надежность при работе с многопоточностью (что критично для стриминга аудиосигналов).

Фреймворк *Spring Boot*. Использование экосистемы *Spring* (*Spring Data JPA*, *Spring Web*) позволило ускорить процесс разработки за счет автоматической конфигурации компонентов, реализовать эффективную архитектуру *REST API* для взаимодействия с клиентской частью и обеспечить изоляцию транзакций при работе с базой данных с помощью декларативного управления (*@Transactional*)[9].

В качестве альтернативы рассматривался *Node.js* (*JavaScript*). Однако для задач, связанных с анализом аудиопотоков (подсчет *BPM*, извлечение аудио-фич), обработкой транзакций и построением отказоустойчивой архитектуры, связка *Java/Spring Boot* является более стабильным и производительным решением.

Для реализации пользовательского интерфейса выбраны стандартные веб-технологии: *HTML5*, *CSS3* и *JavaScript* (*Vanilla JS*) без использования тяжелых фреймворков (класса *React/Angular*).

HTML5 и *CSS3* позволили создать адаптивный, отзывчивый интерфейс в темных тонах (индустриальный стандарт медиаплееров) с использованием современных механизмов верстки (*Flexbox*, *Grid Layout*).

Асинхронное взаимодействие с сервером реализовано через встроенный интерфейс *JavaScript Fetch API*.

Для хранения структурированных данных (информация о пользователях, треках, альбомах и аудио-характеристиках) выбрана реляционная СУБД *PostgreSQL*, потому что она обеспечивает полную поддержку принципов *ACID*, гарантируя сохранность данных, обладает высокой производительностью при выполнении сложных выборки и операций

к базовому окладу. Таким образом полный расчет затрат на основную заработную плату команды разработчиков представлен на таблице 2.

$$Z_o = K_{пр} \sum_{i=1}^n Z_{чи} \cdot t_i, \quad (1)$$

где $K_{пр}$ – коэффициент премий и иных стимулирующих выплат;
 n – категории исполнителей, занятых разработкой программного средства;
 $Z_{чи}$ – часовой оклад плата исполнителя i -й категории, р.;
 t_i – трудоемкость работ, выполняемых исполнителем i -й категории, ч.

Таблица 2 – Расчет затрат на основную заработную плату команды разработчиков

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоемкость работ, ч	Итого, р.
Бизнес-аналитик	4 200,00	25,00	80	2 000,00
Системный архитектор	7 500,00	44,64	60	2 678,40
Программист	6 000,00	35,71	320	11 427,20
Тестировщик	3 800,00	22,62	120	2 714,40
Дизайнер	4 000,00	23,81	100	2 381,00
Итого				21 201,00
Премия и иные стимулирующие выплаты (50%)				10 600,50
Всего затрат на основную заработную плату разработчиков				31 801,50

Процесс вычисления включает в себя суммирование произведений часовых тарифных ставок на соответствующую трудоемкость работ для каждой роли в проекте. Для бизнес-аналитика этот показатель составил 2 000 р., для системного архитектора – 2 678,40 р., для программиста – 11 427,20 р., для тестировщика – 2 714,40 р. и для дизайнера – 2 381,00 р.

Суммарная величина выплат по базовым окладам для всей команды составила 21 201,00 р. После применения установленного коэффициента стимулирующих выплат итоговое значение основной заработной платы разработчиков составило 31 801,50 р. Данная сумма является базой для дальнейшего расчета отчислений в бюджет и оценки общей себестоимости программного продукта.

Для вычисления дополнительной заработной платы разработчиков используется показатель основной заработной платы, рассчитанный ранее, и норматив дополнительной заработной платы H_d . Значение H_d выбирается в диапазоне от 10% до 20%. Для данного проекта примем норматив в размере 10%, что является стандартным значением для учета выплат за непроработанное время (например, отпуска или выполнение государственных обязанностей). Подставляя в формулу 2 значение основной заработной платы команды, равное 31 801,50 р.

Таким образом, сумма дополнительной заработной платы разработчиков составляет 3 180,15 белорусских рублей.

Для расчета отчислений на социальные нужды воспользуемся формулой (3). Данный показатель отражает обязательные платежи в Фонд социальной защиты населения.

$$Z_d = \frac{Z_o \cdot H_d}{100} \quad (2)$$

где H_d – норматив дополнительной заработной платы;
 Z_o – основная заработная плата разработчиков

Согласно методическим указаниям, норматив отчислений составляет 34,6%. Расчет производится от общей суммы фонда оплаты труда, которая включает в себя основную и дополнительную заработную плату разработчиков.

Используя полученные ранее значения, определим итоговую сумму, подставив значения в формулу 3.

$$P_{\text{соц}} = \frac{(Z_o + Z_d) \cdot H_{\text{соц}}}{100}, \quad (3)$$

где $H_{\text{соц}}$ – норматив отчислений в ФСЗН и Белгосстрах.

Таким образом, общая сумма отчислений на социальные нужды составляет 12 103,65 р. Эти средства являются обязательными издержками организации-разработчика и включаются в полную себестоимость программного продукта.

Следующим этапом в расчете затрат является определение прочих расходов по формуле 4. Данная статья затрат включает в себя общехозяйственные и общепроизводственные издержки, связанные с обеспечением процесса разработки.

ПРИЛОЖЕНИЕ А (ОБЯЗАТЕЛЬНОЕ) ЛИСТИНГ КОДА

```
@GetMapping("/{id}/stream")
public ResponseEntity<StreamingResponseBody> streamTrack(
    @PathVariable Integer id,
    @RequestHeader HttpHeaders headers) {
    try {
        StorageService.S3ObjectWrapper s3Object = trackService.getTrackObject(id);
        InputStream inputStream = s3Object.getInputStream();
        long fileLength = s3Object.getContentLength();
        List<HttpRange> ranges = headers.getRange();

        if (ranges.isEmpty()) {
            StreamingResponseBody responseBody = outputStream -> {
                byte[] buffer = new byte[4096];
                int bytesRead;
                try (inputStream) {
                    while ((bytesRead = inputStream.read(buffer)) != -1) {
                        outputStream.write(buffer, 0, bytesRead);
                    }
                }
            };

            return ResponseEntity.status(HttpStatus.OK)
                .header(HttpHeaders.ACCEPT_RANGES, "bytes")
                .header(HttpHeaders.CONTENT_LENGTH, String.valueOf(fileLength))
                .contentType(MediaType.parseMediaType("audio/mpeg"))
                .body(responseBody);
        }

        HttpRange range = ranges.get(0);
        long start = range.getRangeStart(fileLength);
        long end = range.getRangeEnd(fileLength);
        long rangeLength = end - start + 1;
        long skipped = inputStream.skip(start);
        StreamingResponseBody responseBody = outputStream -> {
            byte[] buffer = new byte[4096];
            long bytesToRead = rangeLength;
            try (inputStream) {
                while (bytesToRead > 0) {
                    int maxRead = (int) Math.min(buffer.length, bytesToRead);
                    int bytesRead = inputStream.read(buffer, 0, maxRead);
                    if (bytesRead == -1) break;
                    outputStream.write(buffer, 0, bytesRead);
                }
            }
        };
    }
}
```